

SSH Log Analysis Using Splunk SIEM

SOC Analyst Project Report

Author: Leonard N

Tool: Splunk Enterprise | Log Type: SSH (sshlogs)

Objective	Analyse SSH logs to detect brute-force attacks and compromised accounts
Platform	Splunk Enterprise — Search & Reporting
Source Type	sshlogs
Total Events	7,143 SSH events analysed
Key Findings	Brute-force attack confirmed — 71% failure rate, 301 successful logins
Outcome	Confirmed SSH compromise — incident response escalated

SECTION 1

Introduction

SSH (Secure Shell) is the primary protocol for remote server administration. SSH log files record every login attempt, session event, and authentication outcome — making them one of the richest data sources for detecting intrusions.

In this project I ingested SSH logs into **Splunk Enterprise** and performed a structured investigation: extracting key fields, profiling source IP behaviour, measuring authentication success and failure rates, analysing session patterns, and applying threshold-based detection to classify suspicious hosts. The investigation escalated from anomaly detection to a **confirmed security incident**.

SECTION 2

Step 1 — Field Extraction

Before running any queries I used Splunk's **Extract Fields (IFX)** wizard to parse the raw SSH log events using a regular expression. The extracted fields are the foundation of every subsequent query in this investigation:

- **src_ip** — IP address of the machine initiating the SSH connection
- **src_port** — Source port used by the connecting client
- **dst_ip** — IP address of the SSH server being accessed
- **dst_port** — Destination port — port 22 is standard SSH
- **status** — Authentication outcome: failure / success / undetermined
- **direction** — Connection direction (e.g. INBOUND)
- **client** — SSH client version string (e.g. OpenSSH_5.3p1)
- **server** — SSH server version string

splunk>enterpriseApps

Administrator6 MessagesSettingsActivityHelpFind

Extract Fields

Select SampleSelect MethodSelect FieldsValidateSave

< BackFinish >

Save

Name the extraction and set permissions.

Extractions NameEXTRACT-

src_ip,src_port,dst_ip,dst_port,status,c

Ownerleonardw

Appsearch

Permissions

OwnerAppAll apps

Source typesshlogs

Sample event

1332017793.040000Cr-UTZx1hjVklqFFT11192.168.202.13656815192.168.21.20322failure

INBOUNDSSH-2.0-OpenSSH_5.3p1 Debian-3ubuntu7SSH-2.0-OpenSSH_5.8p1 Debian-1ubuntu3- - -

Fields

src_ip,src_port,dst_ip,dst_port,status,direction,client,server

Regular Expression

^(?:"\\f\\n"\\t){2}(?P<src_ip>["\\f\\n"\\t](?P<src_port>[d+])\\t(?P<dst_ip>["\\f\\n"\\t](?P<dst_port>["\\f\\n"\\t](?P<status>[a-z]+)["\\f\\n"\\t](?P<direction>[w+])\\t(?P<client>["\\f\\n"\\t](?P<server>["\\f\\n"\\t

Figure 1 — Splunk IFX 'Save' step confirming the extraction name, owner (leonardw), source type (sshlogs), and the 8 extracted fields with their regex pattern.

SECTION 3

Step 2 — Identifying Top Source IPs

With fields in place I ran a **top src_ip** query to find which machines were generating the most SSH traffic. Normal environments show an even spread. What I found was the opposite:

```
index=*_ OR index=* sourcetype=sshlogs | top limit=10 src_ip
```

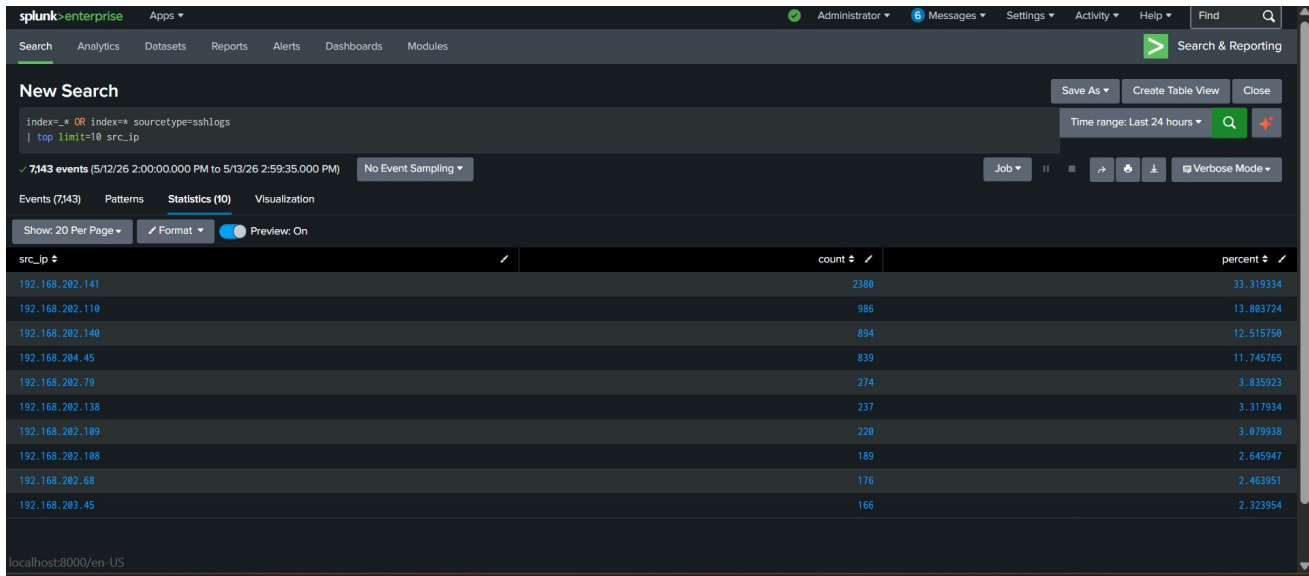


Figure 2 — Top 10 source IPs by SSH event count. 192.168.202.141 leads with 2,380 events (33.3%), and the top 4 IPs together account for ~71% of all activity.

What this tells us:

src_ip	Events	% of Total	Assessment
192.168.202.141	2,380	33.3%	Primary suspect
192.168.202.110	986	13.8%	Suspicious
192.168.202.140	894	12.5%	Suspicious
192.168.204.45	839	11.7%	Suspicious

- Traffic is **highly concentrated** — one host alone accounts for a third of all SSH events.
- All IPs are **internal (192.168.x.x)**, pointing to lateral movement, a compromised internal host, or misconfigured automation.
- This distribution is not normal. Healthy environments show varied, low-count activity spread across many hosts.

SECTION 4

Step 3 — Authentication Success vs Failure

I queried authentication outcomes across all events to measure the failure-to-success ratio. A high failure rate is the clearest indicator of a brute-force attack.

```
index=_* OR index=* sourcetype=sshlogs | stats count by status
```

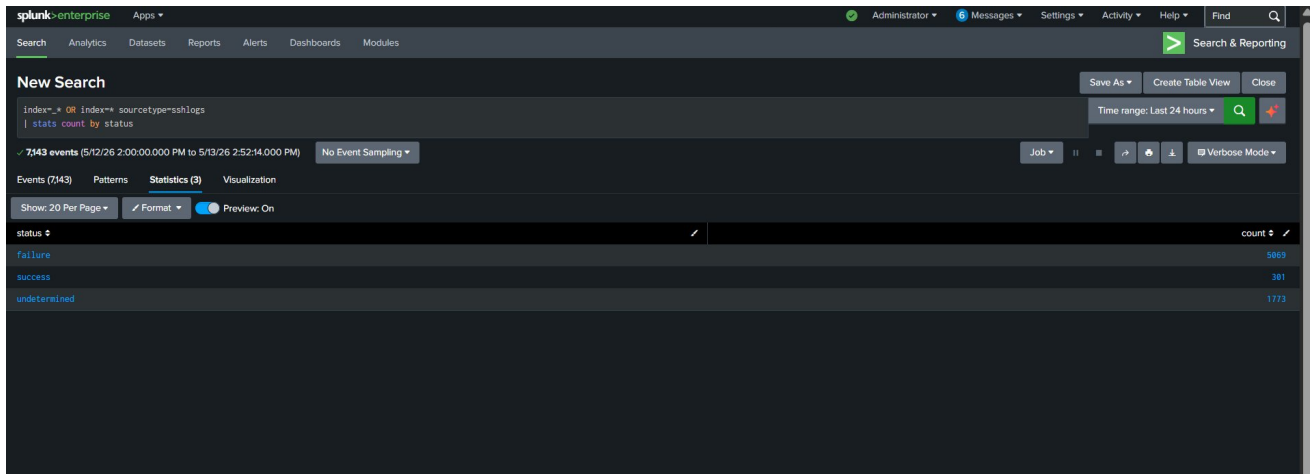


Figure 3 — Authentication status breakdown across 7,143 SSH events.

Results:

Status	Count	Rate	Meaning
Failure	5,069	71%	Failed login attempts — brute-force indicator
Undetermined	1,773	25%	Could not confirm outcome
Success	301	4%	Authenticated logins — potential compromise

Key insight: A 71% failure rate is far outside normal. In a healthy environment you expect the vast majority of SSH events to be successful logins by known users. Here, failures dominate — a textbook brute-force signature. The 301 successful logins are the most concerning part: **some attempts got through**.

Step 4 — Session Duration & SSH Version Analysis

To understand connection behaviour I built a session table grouping events by source IP and SSH server version. This reveals whether hosts are making legitimate interactive sessions or firing automated connection probes.

```
index== sourcetype=sshlogs | eval session_key = src_ip . " : " . server | stats
range(timestamp) as session_duration_seconds, count as event_count,
earliest(timestamp) as start_time, latest(timestamp) as end_time by session_key |
eval duration_minutes = round(session_duration_seconds / 60, 2) | sort -
duration_minutes | rename session_key as "Source IP : Server", duration_minutes as
"Duration (min)" | table "Source IP : Server", event_count, "Duration (min)",
start_time, end_time
```

The screenshot shows the Splunk Enterprise interface with a search results table. The search query is: `index== sourcetype=sshlogs | eval session_key = src_ip . " : " . server | stats range(timestamp) as session_duration_seconds, count as event_count, earliest(timestamp) as start_time, latest(timestamp) as end_time by session_key | eval duration_minutes = round(session_duration_seconds / 60, 2) | sort - duration_minutes | rename session_key as "Source IP : Server", duration_minutes as "Duration (min)" | table "Source IP : Server", event_count, "Duration (min)", start_time, end_time`. The results table shows 7143 events. The columns are: Source IP, event_count, Duration (min), start_time, and end_time. The data shows multiple sessions from various source IPs, with most durations being 'none'.

Source IP	event_count	Duration (min)	start_time	end_time
192.168.202.180 SSH-1.0-Cluster-1.25	5	none	none	none
192.168.202.180 SSH-1.0-OpenSSH_4.5	1	none	none	none
192.168.202.180 SSH-2.0-OpenSSH_4.5	5	none	none	none
192.168.202.180 SSH-2.0-OpenSSH_4.5	13	none	none	none
192.168.202.180 SSH-2.0-OpenSSH_4.5	4	none	none	none
192.168.202.181 SSH-1.0-OpenSSH_4.5	4	none	none	none
192.168.202.181 SSH-1.0-Cluster-1.25	1	none	none	none
192.168.202.181 SSH-2.0-OpenSSH_4.5	1	none	none	none
192.168.202.182 SSH-1.0-Cluster-1.25	14	none	none	none
192.168.202.182 SSH-1.0-OpenSSH_4.5	1	none	none	none
192.168.202.182 SSH-2.0-OpenSSH_4.5	11	none	none	none
192.168.202.182 SSH-2.0-OpenSSH_4.5	11	none	none	none
192.168.202.183 SSH-1.0-OpenSSH_4.5	1	none	none	none
192.168.202.183 SSH-2.0-OpenSSH_4.5	18	none	none	none
192.168.202.183 SSH-2.0-OpenSSH_4.5	6	none	none	none
192.168.202.188 SSH-1.0-Cluster-1.25	26	none	none	none
192.168.202.188 SSH-1.0-OpenSSH_4.5	11	none	none	none
192.168.202.188 SSH-2.0-OpenSSH_4.5	65	none	none	none
192.168.202.188 SSH-2.0-OpenSSH_4.5	62	none	none	none
192.168.202.188 SSH-2.0-OpenSSH_4.5	25	none	none	none

Figure 4 — Session analysis table showing Source IP paired with SSH server version, event count, and duration. Duration values of 'none' indicate no time gap — consistent with rapid automated probing rather than interactive sessions.

What stood out:

- **Multiple SSH version fingerprints** detected per host (OpenSSH 4.5, 5.3p1, 5.8p1, Cisco SSH 1.25) — a scanner systematically trying different SSH implementations.
- **Duration = none** on nearly all sessions — connections open and close instantly, characteristic of automated tooling, not human users.
- **Same internal IPs** appear across different SSH service types, confirming network-wide SSH scanning from compromised internal hosts.

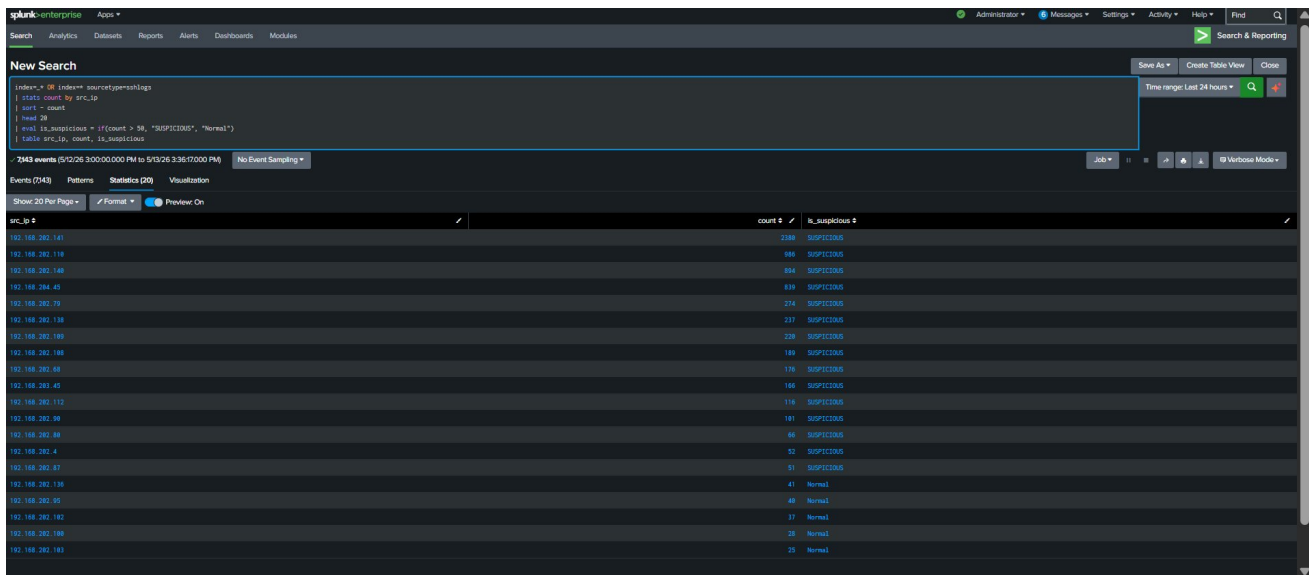
This behaviour is consistent with **SSH reconnaissance and credential stuffing** across the internal network.

SECTION 6

Step 5 — Threshold-Based Suspicious IP Detection

To move from observation to actionable classification I built a detection rule directly in SPL: any source IP generating more than 50 SSH events is flagged as **SUSPICIOUS**. This converts raw counts into a usable alert feed.

```
index=_* OR index=* sourcetype=sshlogs | stats count by src_ip | sort - count | head 20 | eval is_suspicious = if(count > 50, "SUSPICIOUS", "Normal") | table src_ip, count, is_suspicious
```



The screenshot shows the Splunk Enterprise interface with a search bar containing the SPL query: `index=_* OR index=* sourcetype=sshlogs | stats count by src_ip | sort - count | head 20 | eval is_suspicious = if(count > 50, "SUSPICIOUS", "Normal") | table src_ip, count, is_suspicious`. The search results are displayed in a table with columns: `src_ip`, `count`, and `is_suspicious`. The table shows 20 rows of data, with the first 15 rows classified as `SUSPICIOUS` and the last 5 rows as `Normal`.

src_ip	count	is_suspicious
192.168.202.141	2380	SUSPICIOUS
192.168.202.118	986	SUSPICIOUS
192.168.202.148	884	SUSPICIOUS
192.168.204.45	839	SUSPICIOUS
192.168.202.79	274	SUSPICIOUS
192.168.202.138	237	SUSPICIOUS
192.168.202.105	228	SUSPICIOUS
192.168.202.108	180	SUSPICIOUS
192.168.202.68	179	SUSPICIOUS
192.168.202.45	165	SUSPICIOUS
192.168.202.112	119	SUSPICIOUS
192.168.202.98	101	SUSPICIOUS
192.168.202.88	89	SUSPICIOUS
192.168.202.4	52	SUSPICIOUS
192.168.202.87	51	SUSPICIOUS
192.168.202.136	41	Normal
192.168.202.95	40	Normal
192.168.202.162	37	Normal
192.168.202.100	25	Normal
192.168.202.183	25	Normal

Figure 5 — Threshold detection output. Every IP exceeding 50 events is labelled SUSPICIOUS. 15 out of 20 top hosts are flagged, several with counts in the hundreds.

What this confirms:

- **15 IPs** exceed the 50-event threshold and are classified SUSPICIOUS — this is systematic, not random noise.
- The top IP (192.168.202.141) has 2,380 events — **47x** the detection threshold.
- Combined with the 71% failure rate and 301 successes, this confirms: **an active, automated SSH brute-force campaign with confirmed successful logins.**

Confirmed Incident — SOC Response Actions

The evidence across all five analysis steps escalates this from a suspicious alert to a **confirmed SSH authentication compromise**. Multiple internal IPs conducted high-volume brute-force attacks and at least one credential set was successfully compromised. The following response actions are required:

1	Immediate Containment Isolate all hosts associated with successful SSH logins. Block all flagged source IPs at the firewall and SIEM blocklist. Restrict SSH access on impacted systems immediately.
2	Identity & Access Response Disable all accounts linked to successful SSH events. Force-terminate active sessions. Rotate all passwords, SSH keys, and service credentials on affected systems. Prioritise privileged (root/admin) accounts.
3	Incident Scoping Map every successful login to a user account, target system, and timestamp. Build a full attack timeline from first failure through to successful authentication and post-login activity. Check for lateral movement across the internal network.
4	Threat Validation Confirm whether post-login actions occurred: privilege escalation, new account creation, suspicious processes, or file modifications. Check for persistence mechanisms (cron jobs, SSH authorized_keys changes, backdoors).
5	Eradication Remove all compromised credentials and unauthorised configurations. Rebuild or reimage systems where integrity cannot be guaranteed. Patch the initial access vector — enforce key-based auth, disable password auth.
6	Recovery & Hardening Restore systems only after confirming no active sessions or persistence exist. Enforce MFA for SSH, restrict access to VPN-only environments, implement rate limiting and fail2ban-style controls.
7	Detection Improvement Tune Splunk alerts to trigger on high failure-then-success patterns. Implement risk scoring by combining event frequency, source reputation, and success-after-failure sequences. Enable real-time alerting for brute-force patterns.

Conclusion

This investigation demonstrates a complete SOC analyst workflow applied to SSH logs. Starting from raw log ingestion and field extraction, I progressively built a picture of the threat — identifying abnormal IP concentration, confirming a 71% authentication failure rate, detecting multiple SSH version fingerprints indicating automated scanning, and applying threshold rules to classify 15 suspicious hosts.

The 301 successful logins in the context of a brute-force campaign represent a **confirmed security incident**. At least one set of credentials was compromised. Full post-authentication investigation is required to determine whether lateral movement or persistence occurred, followed by credential rotation, system hardening, and enhanced real-time monitoring.

Splunk's SPL query language and field extraction capabilities proved essential for rapidly processing over 7,000 events and surfacing actionable findings that would be impossible to detect through manual log review.